

Agenda for Today

- Announcements
- Quick comments on midterm
- `def`, `while`, `continue`, `break`
- bits and bytes
- Characters and colours using ASCII and Hexadecimal RGB

Announcements

Reminder: Python Functions with `def`

- `def` : Used to define a function in Python

Syntax:

```
def function_name(parameters):  
    # code block  
    return True
```

Example:

```
def greet(name):  
    return f"Hello, {name}!"
```

- Functions:
 - Encapsulate code for reuse
 - Can accept parameters
 - Can return values (or `None` if no return statement)

Three more examples

Ex1:

```
def foo1(a,b):  
    return a+b
```

Ex2:

```
def foo2(n):  
    return n % 2 == 0
```

Ex3:

```
def foo3(n):  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result
```

Better names

Ex1:

```
def add_two_nums(a,b):  
    return a+b
```

Ex2:

```
def is_even(n):  
    return n % 2 == 0
```

Ex3:

```
def factorial(n):  
    result = 1  
    for i in range(1, n + 1):  
        result *= i  
    return result
```

Python **while** Loop

- **while** loop: Repeats a block of code as long as a condition is **True**

Syntax:

```
while condition:  
    # code block
```

Example:

```
i = 0  
while i < 5:  
    print(i)  
    i += 1
```

Why Use a `while` Loop (or not!)?

- **Good for: Flexible Condition:**
 - Use `while` when the number of iterations is unknown.
 - Example: Waiting for user input or external event.
 - When the loop condition is more complex than simple iteration (e.g., event-based, real-time updates).

Example:

```
while not user_input:  
    user_input = input("Enter something: ")
```

- **Watch out for:**

- Infinite Loops: Easier to create loops that run indefinitely until a condition is met.
- Probably don't need while loops when you can define a specific range or collection to iterate over.

Python `continue`

- `continue` : Used inside a loop. Skips the rest of the code inside the loop for the current iteration and moves to the next iteration.
- **Use Case:**
 - When you want to skip certain iterations based on a condition.

Syntax:

```
while condition:  
    if some_check:  
        continue  
    # remaining code for other iterations
```

Example:

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

Output: 0, 1, 3, 4 (skips 2)

- **Key Point:**

- Useful for skipping specific cases without stopping the loop.
- Sometime if you "found what you're looking for" you can continue

More Practical Examples of `continue`

Skip Even Numbers:

```
for i in range(10):  
    if i % 2 == 0:  
        continue  
    print(i)
```

- Output: `1, 3, 5, 7, 9` (skips even numbers)

Skip Blank Input:

```
inputs = ["hello", "", "world", "", "python"]
for item in inputs:
    if item == "":
        continue
    print(item)
```

Output: hello, world, python (skips empty strings)

- **Key Takeaway:**

- `continue` helps ignore unwanted cases, allowing the loop to focus on relevant data.

Python **break**

- **break** : Immediately exits the loop, stopping further iterations.
- **Use Case:**
 - When you need to stop the loop based on a condition.
- **Syntax:**

```
while condition:  
    if some_check:  
        break  
    # remaining code
```

Example of break:

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

Output: 0, 1, 2, 3, 4 (loop stops at i == 5)

- **Key Point:**

- `break` is useful when you want to exit a loop early based on a condition.

Practical Examples of `break`

Stop on User Input:

```
while True:
    user_input = input("Enter a number (or 'q' to quit): ")
    if user_input == 'q':
        break
    print(f"You entered: {user_input}")
```

- The loop stops when the user enters `q`.

Find First Match in a List:

```
numbers = [3, 5, 7, 8, 10, 12]
for num in numbers:
    if num % 2 == 0:
        print(f"First even number found: {num}")
        break
```

Summary of our control flow tools

- we already know `if`, `for`, and `range` really well
- now we know `def`, `while`, `continue`, and `break`
- These are our core control flow tools for CMPT 120
- If you think you'll write a lot of Python, consider just reading the official docs now:
<https://docs.python.org/3/tutorial/controlflow.html>
 - These are also useful if you just want to read the sections about `if`, `for`, `range`, `def`, `while`, `continue`, `break`

Connecting back to bits and bytes

Let's put it all together and transition back to bits and bytes!

We'll work through a longer code example that uses `def`, `while`, `continue`, and `break`.

Task: write a program that takes an 8-bit binary number from user and prints out the answer in decimal. Now, are program will continue to run until user enters 'exit' (while loop + break) and will ask the user to fix their input if it is invalid (continue).

Step 1: Write a function to convert binary to decimal

```
def binary_to_decimal(binary_str):  
    """Convert a binary string to a decimal number."""  
    decimal = 0  
    for digit in binary_str:  
        decimal = decimal * 2 + int(digit)  
    return decimal
```

Step 2: Write a function to convert decimal to ascii

(Turns out we can just use a Python built-in: effectively we're just giving it a new name)

```
def decimal_to_ascii(decimal):  
    """Convert a decimal number to its ASCII character."""  
    return chr(decimal)
```

Step 3: Use a while loop with break and continue

```
def process_binary_input():  
    """Process binary input and handle conversion to decimal and ASCII."""  
    while True:  
        binary_input = input("Enter an 8-bit binary number (or 'exit' to quit): ")  
  
        if binary_input.lower() == 'exit':  
            print("Exiting...")  
            break  
  
        # Ensure input is 8 bits and binary  
        if len(binary_input) != 8 or not all(bit in '01' for bit in binary_input):  
            print("Invalid input! Please enter an 8-bit binary number.")  
            continue  
  
        # Convert binary to decimal  
        decimal_value = binary_to_decimal(binary_input)  
  
        # Convert decimal to ASCII  
        ascii_char = decimal_to_ascii(decimal_value)  
  
        print(f"Binary: {binary_input} -> Decimal: {decimal_value} -> ASCII: {ascii_char}")  
  
# Call the function to start the loop  
process_binary_input()
```

How to Read Binary Code

Converting Binary to Decimal

- **Binary Representation:**

- Binary is a base-2 system (only 0 and 1).
- Each binary digit (bit) represents a power of 2.
- Rightmost digit is 1.
- Second from right is 2.
- Third from right 4. And so on.

Example:

Binary: 1011

$$1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0$$

11

- **Start from the rightmost bit:**
 - Assign powers of 2 (starting from 2^0).
- **Multiply each bit** by its corresponding power of 2.
- **Sum the results** to get the decimal number.

Reading Binary

Power of 2	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
Value	128	64	32	16	8	4	2	1
Binary	1	0	1	1	0	1	0	1

Total: $128 + 0 + 32 + 16 + 0 + 4 + 0 + 1 = 181$

What is ASCII?

- American Standard Code for Information Interchange
- ASCII is a table that maps decimal numbers to characters

ASCII , some random examples

Character	Binary	Decimal	Hexadecimal	Description
A	0100 0001	65	0x41	Uppercase A
a	0110 0001	97	0x61	Lowercase a
0	0011 0000	48	0x30	Number zero
	0010 0000	32	0x20	Space
!	0010 0001	33	0x21	Exclamation
\n	0000 1010	10	0x0A	New line

Note: ASCII uses 7 bits, allowing for 128 characters (0-127).

Extended ASCII uses 8 bits, allowing for 256 characters (0-255).

ASCII Table Sequence

Dec	Hex	Char	Description
32	0x20		Space
33	0x21	!	Exclamation
34	0x22	"	Double quote
35	0x23	#	Hash
36	0x24	\$	Dollar
37	0x25	%	Percent
38	0x26	&	Ampersand
39	0x27	'	Single quote
40	0x28	(	Open paren
41	0x29	)	Close paren

Python Examples

```
# Get ASCII code from character
print(ord('A')) # Output: 65
print(ord('!')) # Output: 33

# Get character from ASCII code
print(chr(65))  # Output: A
print(chr(33))  # Output: !

# String of ASCII characters
print(''.join(chr(i) for i in range(65, 91))) # A-Z
```

Decode the Binary Message!

01001000 01101001 00100001 00100000 01011001 01101111 01110101 00100000
01110010 01101111 01100011 01101011 00100001

💡 Hint: Each character is 8 bits (1 byte)

Could try grabbing scratch paper and looking up the ASCII table or...

Or: convert 01001000 -> 72 -> H.

```
def binary_to_decimal(binary_str):  
    """Convert a binary string to a decimal number."""  
    decimal = 0  
    for digit in binary_str:  
        decimal = decimal * 2 + int(digit)  
    return decimal  
  
binary_to_decimal("01001000")  
chr(72)
```

Solution: Hi! You rock!

Hexadecimal

Hexadecimal is a numbering system that's often used for RGB colors.

- Binary uses powers of 2, so we only need two characters (0 and 1)
- Decimal uses power of 10, so we need ten characters
- Hexadecimal uses power of 16...

But we only have 10 digits?

Taking 6 extra characters from the alphabet

In hexadecimal, there are 16 digits (we use A - F for numbers 11-16).

- 0000 (binary) <> 0 (decimal) <> 0 (hexadecimal)
- 0001 (binary) <> 1 (decimal) <> 1 (hexadecimal)
- 0010 (binary) <> 2 (decimal) <> 2 (hexadecimal)

Ok, it's all the same until...

- 1010 (binary) $\leftrightarrow$ 10 (decimal) $\leftrightarrow$ A (hexadecimal)
- 1011 (binary) $\leftrightarrow$ 11 (decimal) $\leftrightarrow$ B (hexadecimal)
- 1100 (binary) $\leftrightarrow$ 12 (decimal) $\leftrightarrow$ C (hexadecimal)
- 1101 (binary) $\leftrightarrow$ 13 (decimal) $\leftrightarrow$ D (hexadecimal)
- 1110 (binary) $\leftrightarrow$ 14 (decimal) $\leftrightarrow$ E (hexadecimal)
- 1111 (binary) $\leftrightarrow$ 15 (decimal) $\leftrightarrow$ F (hexadecimal)

(TLDR: A = 10, B = 11, and so on)

Example of hexadecimal

00FFAA is a green-blue color

00 means "0 red"

FF means "255 Green"

AA means "170 blue"

Why is FF 255?

$$\text{FF} \rightarrow 15 * (16 ** 1) + 15 (16 ** 0) = 225 + 16 = 240 + 15 = 255$$

Importance of Grouping

If you just know that 1 byte is 8 bits, that binary codes used powers of 2, and that and hexadecimal uses powers of 16 and the characters A-F, you can solve a lot of problems from "first principles".

However we also need to learn common practices around grouping:

- when storing text using an 8-bit character set like ASCII, bits are grouped into 8 bits so a string like "01001000 01101001" can be converted to two characters
- when storing colors as RGB hexadecimal strings, we only need 4 bits per character (only 16 possible characters)
- How many bytes for the string "AB"? 8 times 2 equals 16
- How many bytes for the color 00FFAA? 4 times 6 equals 24.

For fun: see Wikipedia on Unicode and Mojibake <https://en.wikipedia.org/wiki/Mojibake>

One more example connecting back to our coding skills

```
dec = ["65", "66", "67"]
chars = ["A", "B", "C"]
characters_as_dec = ["65", "65", "65", "67", "67", "67"]
for i in range(len(characters_as_dec)):
    for j in range(len(dec)):
        if characters_as_dec[i] == dec[j]:
            print(chars[j])
```

Outcomes

What do we need to know here?

1. Knows what a bit represents in a computer
2. Convert a given number of bytes (or kb, mb) to bits
3. Convert binary $\leftrightarrow$ decimal
4. Understand the goal of hexadecimal
5. Know that binary numbers convert to hexadecimal by grouping of 4 bits
6. Know the purpose of ASCII
7. Know how to store RGB numbers (3 byte aka 24 bits aka 6 hexadecimal digits)

Agenda for Part 2

- Revisit functions (add a mental model)
- Turtle: drawing in Python
- Quick while loop refresh
- Your questions or extra practice (time permitting)

Recap, Recipe for a Function

To create a function, follow these steps:

1. Use `def` + `function_name` + arguments in parentheses + a colon.
2. Indent all code within the function block.
3. Use `return` to send back a value.

Example:

```
def greet(name):  
    message = f"Hello, {name}!"  
    return message
```

Calling `greet("Alice")` will return `"Hello, Alice!"`.

Scope in Functions

Functions have their own scope. Variables defined inside functions aren't accessible outside them unless returned.

Tip: Imagine your function as if it were in a separate file.

Try It: Visualize Functions

- Open the Python Tutor visualizer and paste one of the examples.
- Step through execution to see variables created inside the function scope.
- Predict each return value before you run it, then verify your guess.

Default Arguments

You can set default values for arguments:

```
def power(base, exponent=2):  
    return base ** exponent
```

Using `power(3)` defaults to `exponent=2`, giving `9`, while `power(3, 3)` returns `27`.

Challenge: Quick Default Argument Check-In

- Think of a built-in function we call with and without optional arguments (`range` , `print` , `round` ...).
- Why do defaults matter? They let us skip parameters until we actually need to change them.
- We'll practice by repairing a broken script that misuses arguments.

Mini Challenge: Patch the Functions

- Only add code; don't delete or reorder anything.
- Add sensible default values and fix argument counts so every call succeeds.
- Predict the output before running the test harness.

```
# Try to get this code to run by ONLY ADDING things
```

```
# give this a default argument
```

```
def f1(x):  
    return x * 2
```

```
# takes multiple arguments: pass them below
```

```
def f2(a, b):  
    return a + b
```

```
# f3 takes no arguments
```

```
def f3(): # No arguments  
    return "Hello!"
```

Test Harness

- Run this block after your fixes to confirm everything works.
- When all tests pass, you should see four success messages.

```
successes = []  
try:  
    r1 = f1()  
    successes.append("f1 works")  
except Exception as e:  
    print(e)  
  
try:  
    r2 = f2()  
    successes.append("f2 works")  
except Exception as e:  
    print(e)  
  
try:  
    r3 = f3("a")  
    successes.append("f3 works")  
except Exception as e:  
    print(e)  
  
try:  
    r4 = f4()  
    successes.append("f4 works")
```

Function Recap

- Define functions with `def`, indent every line inside, and keep names descriptive.
- Respect function scope: pass values in, return results out, avoid hidden globals.
- Use docstrings or comments to capture intent before you forget it.
- Default arguments make functions flexible—pick defaults that match the most common call.
- Troubleshoot by calling each function in isolation (or with a tiny test harness).

Quick While Loop Tune-Up

- Start with a clear condition: “keep looping while...” (usually tied to a variable you control).
- Update that variable inside the loop or you risk an infinite loop.
- Combine `continue` for “skip this case” and `break` for “we’re done here”.
- Pair every `input()` with validation so bad data triggers a retry rather than a crash.
- Add a “sentinel” option like `exit` to give your users an escape hatch.

Mental Model: Buckets & Robots

- Variables are buckets—each holds one value at a time (numbers, strings, lists...).
- Functions are little robots that read buckets, do work, and hand you results.
- If a robot needs new information, pass it in as an argument instead of letting it rummage around.
- When a robot finishes, whatever it `return`s is what goes back into a bucket.

Turtle TLDR Overview

Now, let's do a "Too Long, Didn't Read" tutorial on Python's Turtle graphics.

- This complements the live demo—use it as a quick reference while you code.

How to Use This Deck

Here's the suggested approach for learning:

1. Glance through the slides for an overview.
2. Uncomment and run each code line in `turtle_tldr.py` incrementally to see how Turtle behaves.
3. Experiment by changing function arguments to see their effects on the output.

Turtle TLDR Code Snippets

Here are key code snippets from `turtle_tldr.py` to help you get started:

```
import turtle

# Create a turtle object named pen
pen = turtle.Turtle()
pen.speed(0) # Fastest speed

# Draw a triangle with stamps at each vertex
for i in range(3):
    pen.forward(50)
    pen.stamp()
    pen.left(120)

# Re-position pen for the next part of the drawing
pen.penup()
pen.goto(100, -100)
pen.pendown()

# Draw a square with blue color
pen.color("blue")
for i in range(4):
```

Turtle Practice Prompt

Assignment: Create a Unique Abstract Art Piece

Use Python's Turtle module and the `random` module to generate different abstract artwork with each script run.

Suggested Steps

Try out the following approaches:

- Draw random lines
- Draw random circles and lines
- Draw random squares
- **Advanced:** Use a math function like `sin` (via `import math -> math.sin()`)

Be creative and experiment with Turtle methods like `forward()`, `right()`, `left()`, `penup()`, `pendown()`, `color()`, and `speed()`.

Coding Tips

To organize your code:

- Create a `main` function for the main body of code, and call it at the end with `main()`.

Use this snippet as a starter kit:

```
# Starter kit 5-line snippet for turtle art
import turtle as t
import random
def main():
    t.forward(100)
main()
```

Hints for Getting Started

If you're stuck, try drawing random lines with this function:

```
import random

def draw_random_lines(num_lines):
    for _ in range(num_lines):
        t.color(random.random(), random.random(), random.random())
        t.penup()
        t.goto(random.randint(-200, 200), random.randint(-200, 200))
        t.pendown()
        t.setheading(random.randint(0, 360))
        t.forward(random.randint(50, 200))
```

Call `draw_random_lines(5)` under `main()` to draw five random lines. Experiment with circles or squares from here!

Advanced Color and Positioning

```
# Use RGB colors for more variety
turtle.colormode(255)
mycolor = (255, 0, 120)
pen.color(mycolor)
for _ in range(4):
    pen.forward(100)
    pen.right(90)

# Hide the turtle at the end
pen.hideturtle()
turtle.done()
```

Turtle Recap

- Experiment with different shapes and helper functions to keep code readable.
- Use randomness to create unique outputs—seed with `random.seed()` if you need

Turtle Troubleshooting Tips

- Turtle opens a separate window—run scripts locally (not in some online editors).
- If the window flashes and closes, call `turtle.done()` or `screen.exitonclick()` at the end.
- When nothing draws, double-check that you lifted the pen with `penup()` or moved the turtle off-screen.
- Slow things down with `pen.speed(3)` when you need to watch motion carefully.